Task

Implement `genetic-swarm-ops`: a Trae IDE-compatible executable repository for the **Genetic Swarm Business Operating System** described in `structure.md`, using the IDE/project environment contract described in `starter.md`.

This is not a documentation-only task. Create a working project with files, scripts, tests, Trae workspace outputs, manifests, business operating-system scaffolding, validators, and bootstrap commands.

---

1. Source-of-Truth Documents

Read both files fully before implementing anything:

1. `starter.md`
   Environment and repository contract. It defines the Trae-only starter repository, required scripts, manifests, source download/audit pipeline, security checks, sync layer, tests, and acceptance criteria.

2. `structure.md`
   Business/product architecture. It defines the Genetic Swarm Business Operating System: process intelligence, knowledge acquisition, hybrid memory, workflow DNA, sandboxed evolution, governance, security, evaluation, and agent roster.

Use `starter.md` as the implementation environment contract.

Use `structure.md` as the business/domain architecture contract.

If the two files appear to conflict, follow the stricter safety, auditability, governance, and security requirement.

1.1 Execution State

`starter.md` has already been executed successfully in this repository.

Treat the starter layer as an existing baseline, not as pending work.

Do **not** rerun starter bootstrap steps by default.

Only rerun starter-layer commands if:

- the human explicitly requests a rerun,
- starter-layer files were changed and need revalidation,
- or a later business-layer change depends on updated starter outputs.

Starter-layer baseline already completed:

- repository scaffold created,
- `package.json` created,
- starter manifests created,
- starter scripts implemented,
- Trae workspace outputs generated,

- source download completed,
- `sources/source-lock.json` generated,
- `docs/source-audit.md` generated,
- starter validation commands passed.

---

2. Project Metadata

Use these values unless explicitly overridden by the human:

Field	Value
Project name	`genetic-swarm-ops`
Package name	`genetic-swarm-ops`
Starter spec version	`1.2.0-trae-only`
Business spec version	`2.1`
Purpose	Governed, auditable, self-improving multi-agent business operating system for Trae IDE
Stack	Node.js 20+, plain JavaScript, Node built-ins only for bootstrap scripts
Agent support	Trae IDE + Grok Build (dual-harness)
Package manager	npm
License	MIT

Do not generate editor-specific outputs for Claude Code, Cursor, Gemini, Codex, Copilot, or OpenCode unless they are only downloaded/audited as reference sources per `starter.md`.

---

3. Critical Instructions

You must create an executable repository.

Do not merely summarize `starter.md` or `structure.md`.

You must:

1. Create the complete starter repository structure required by `starter.md`.
2. Copy both `starter.md` and `structure.md` into the project root.
3. Create `package.json`.
4. Create all required manifests.
5. Implement all required Node.js scripts.
6. Create Trae rules, agents, commands, settings, and docs.
7. Download all enabled upstream GitHub sources listed in `sources/manifest.json`, unless network or shell access is unavailable.
8. Generate `sources/source-lock.json`.

9. Generate `docs/source-audit.md`.
10. Create the business operating-system folder structure from `structure.md`.
11. Add business schemas, examples, validators, governance templates, security templates, evaluation templates, and evolution sandbox scaffolding.
12. Add tests for both the starter layer and the business operating-system layer.
13. Run the required validation commands.
14. Update `status.md` with real results.
15. Report final pass/fail status with exact blockers.

If network or shell execution is unavailable, still create all project files and write this exact blocker into `status.md`:

```
BLOCKED: cannot download sources because network/shell execution is unavailable.
```

Do not pretend downloads happened.

---

4. Implementation Mode

If the current directory is empty or is already the intended project root, implement in place.

If the current directory is not empty and does not already appear to be this project, create a new directory:

```
genetic-swarm-ops/
```

Do not overwrite a non-empty target directory unless the human explicitly approved it or passed a force/overwrite instruction.

Preserve user-created files whenever possible.

---

5. Required Starter Layer

The full `starter.md` contract has already been implemented and validated in this repository.

Do not recreate or rerun the starter layer unless one of the conditions in section 1.1 applies.

Use the current starter layer as the fixed project baseline for the remaining business-operating-system work.

The following starter-layer artifacts already exist and should be preserved:

```
starter.md
structure.md
README.md
AGENTS.md
package.json
task.md
```

```
status.md
.gitignore
.editorconfig
.trae/
sources/
external/
scripts/
rules/
skills/
hooks/
mcp-configs/
memory/
reviews/
suggestions/
docs/
examples/
tests/
```

The `package.json` must be an equivalent superset of the one required by `starter.md`.

It must include at least:

```
{
  "type": "module",
  "engines": {
    "node": ">=20.0.0"
  },
  "scripts": {
    "bootstrap": "node scripts/project-starter.mjs bootstrap",
    "create": "node scripts/project-starter.mjs create",
    "init": "node scripts/project-starter.mjs init",
    "doctor": "node scripts/doctor.mjs",
    "sources:download": "node scripts/source-download.mjs",
    "sources:update": "node scripts/source-download.mjs --update",
    "sources:check": "node scripts/source-download.mjs --check",
    "sources:audit": "node scripts/source-audit.mjs",
    "sync": "node scripts/sync.mjs",
    "sync:check": "node scripts/sync.mjs --check",
    "security": "node scripts/security.mjs",
    "review": "node scripts/review.mjs",
    "test": "node --test tests/*.test.mjs"
  }
}
```

Add the business scripts listed in section 9 of this task without regressing the existing starter scripts.

Use Node built-ins only unless the human explicitly approves dependencies.

---

6. Required Source Manifests and Download Pipeline

Create `sources/manifest.json` exactly as specified in `starter.md`.

Create `sources/docs-manifest.json` exactly as specified in `starter.md`.

Implement:

```
scripts/source-download.mjs
scripts/source-audit.mjs
scripts/doctor.mjs
scripts/security.mjs
scripts/sync.mjs
scripts/project-starter.mjs
scripts/create-project.mjs
scripts/review.mjs
scripts/adapters/trae.mjs
scripts/lib/fs-safe.mjs
scripts/lib/git.mjs
scripts/lib/manifest.mjs
scripts/lib/report.mjs
```

The downloader must:

1. Read `sources/manifest.json`.
2. Select all sources with `"enabled": true`.
3. Clone missing repos into `external/sources/`.
4. Update existing git repos with fetch + fast-forward pull.
5. Never run install scripts or downloaded code.
6. Record metadata in `sources/source-lock.json`.
7. Fail on required-source failure.
8. Record optional-source failure without failing unless `--strict` is used.
9. Refuse to write outside the project root.

The audit script must generate:

```
docs/source-audit.md
```

Downloaded repositories are untrusted reference inputs only.

Do not automatically import code, skills, hooks, commands, or MCP servers from downloaded repos into active Trae config.

---

7. Required Business Operating-System Structure

Create the business architecture folder structure from `structure.md`:

```
business/
├── process-intelligence/
│   ├── event-logs/
│   ├── discovered-processes/
│   ├── conformance-reports/
│   ├── bottlenecks/
│   └── causal-hypotheses/
├── knowledge-base/
│   ├── rules/
│   ├── decision-patterns/
│   ├── exceptions/
│   ├── best-practices/
│   ├── tacit-knowledge/
│   └── provenance/
├── experts/
│   ├── profiles/
│   ├── shadow-logs/
│   ├── decision-requirement-cards/
│   └── interview-transcripts/
├── materials/
│   ├── documents/
│   ├── regulations/
│   └── sops/
├── distilled/
│   ├── skills/
│   ├── prompts/
│   ├── workflows/
│   ├── checklists/
│   └── playbooks/
├── memory/
│   ├── episodic/
│   ├── semantic/
│   ├── procedural/
│   ├── decision-memory/
│   └── evaluation-memory/
├── evals/
│   ├── golden-tasks/
│   ├── regression-tests/
│   ├── adversarial-tests/
│   ├── human-review-sets/
│   └── benchmark-results/
├── governance/
│   ├── ai-inventory/
│   ├── use-case-risk-tiering/
│   ├── risk-assessments/
│   ├── human-approval-policy/
│   ├── audit-logs/
│   ├── model-cards/
│   └── assurance-cases/
├── security/
│   ├── threat-models/
│   ├── tool-permissions/
│   ├── prompt-injection-tests/
```

```
│   ├── red-team-results/
│   └── incident-reports/
└── evolution/
    ├── workflow-dna/
    ├── successful-variants/
    ├── failed-experiments/
    ├── mutation-history/
    └── lessons-learned/
```

Also create implementation support directories:

```
business/schemas/
business/examples/
business/policies/
business/adapters/
business/reports/
```

These support directories are allowed extensions for executable validation and documentation.

---

8. Required Business Artifacts

Create machine-readable schemas, human-readable templates, and examples for the core artifacts in `structure.md`.

8.1 Event Log

Create:

```
business/schemas/event-log.schema.json
business/examples/event-log.example.json
business/process-intelligence/event-logs/README.md
```

The event-log schema must validate, at minimum:

- `id`
- `timestamp`
- `actor_type`
- `actor_id`
- `process_id`
- `case_id`
- `activity`
- `input_refs`
- `output_refs`
- `tools_used`
- `decision_point`

- `decision_reason_summary`
- `confidence`
- `risk_tier`
- `human_approved`
- `outcome.status`
- `outcome.latency_minutes`
- `outcome.quality_score`

Use these allowed `actor_type` values:

```
human
agent
system
```

Use these allowed risk tiers:

```
tier_0_observe
tier_1_recommend
tier_2_draft
tier_3_execute_reversible
tier_4_execute_with_gate
tier_5_restricted
```

8.2 Decision Requirement Card

Create:

```
business/schemas/decision-requirement-card.schema.json
business/examples/decision-requirement-card.example.json
business/experts/decision-requirement-cards/README.md
```

The schema must validate, at minimum:

- `id`
- `domain`
- `decision_point`
- `expert_sources`
- `context_signals`
- `cues_experts_notice`
- `normal_action`
- `exception_paths`
- `red_flags`
- `required_evidence`
- `risk_tier`

- `human_approval_required`
- `validation_tests`
- `confidence`
- `last_reviewed`

8.3 Workflow DNA

Create:

```
business/schemas/workflow-dna.schema.json
business/examples/workflow-dna.example.json
business/evolution/workflow-dna/README.md
```

The schema must validate, at minimum:

- `id`
- `name`
- `domain`
- `objective`
- `owner`
- `version`
- `inputs`
- `preconditions`
- `steps`
- `memory_reads`
- `memory_writes`
- `guardrails`
- `verification.required_checks`
- `rollback`
- `fitness_metrics`

Every workflow must include:

- bounded state-graph style steps,
- explicit agents,
- explicit tools,
- memory read/write declarations,
- guardrails,
- verification checks,
- rollback plan,
- fitness metrics,
- audit-log write requirement.

The validator must fail any production workflow that does not include human gates for high-risk, irreversible, or exception-handling actions.

8.4 Evaluation Card

Create:

```
business/schemas/evaluation-card.schema.json
business/examples/evaluation-card.example.json
business/evals/README.md
```

The schema must validate, at minimum:

- `target`
- `eval_type`
- `test_set`
- `metrics`
- `result`
- `promotion_decision`
- `reviewer`

Required metric concepts:

- quality score,
- compliance pass rate,
- average cycle time,
- escalation rate,
- hallucination rate,
- unauthorized tool attempts,
- cost per case.

8.5 Governance Artifacts

Create templates or seed files for:

```
business/governance/ai-inventory/README.md
business/governance/use-case-risk-tiering/risk-tiers.json
business/governance/risk-assessments/README.md
business/governance/human-approval-policy/policy.md
business/governance/audit-logs/README.md
business/governance/model-cards/model-card.template.md
business/governance/assurance-cases/assurance-case.template.md
```

The autonomy risk tiers must match `structure.md`:

Tier	Meaning
0	Observe only
1	Recommend only
2	Draft, human approves

Tier	Meaning
3	Execute reversible low-risk actions
4	Execute with human gate for critical step
5	Restricted; no autonomous action until assurance case exists

8.6 Security Artifacts

Create:

```
business/security/threat-models/threat-model.template.md
business/security/tool-permissions/tool-permission-register.json
business/security/prompt-injection-tests/README.md
business/security/red-team-results/README.md
business/security/incident-reports/incident-report.template.md
```

Security controls must cover:

- indirect prompt injection,
- sensitive information disclosure,
- supply-chain risk,
- memory poisoning,
- improper output handling,
- excessive agency,
- prompt leakage,
- vector/embedding weaknesses,
- misinformation,
- unbounded consumption,
- tool misuse,
- identity and privilege abuse.

8.7 Evolution Sandbox

Create:

```
business/evolution/README.md
business/evolution/workflow-dna/README.md
business/evolution/successful-variants/README.md
business/evolution/failed-experiments/README.md
business/evolution/mutation-history/README.md
business/evolution/lessons-learned/README.md
```

The evolution layer must enforce this non-negotiable rule:

```
The Evolution Manager must never mutate production directly.
It may only propose variants, test in sandbox, compare to baseline,
request approval, canary deploy, and rollback on failure.
```

First implementation may create the sandbox framework and validators rather than a full genetic optimization engine.

---

9. Required Business Scripts

Add these scripts to `package.json`:

```
{
  "business:init": "node scripts/business/init-business.mjs",
  "business:validate": "node scripts/business/validate-business.mjs",
  "business:eval": "node scripts/business/eval-harness.mjs",
  "business:governance": "node scripts/business/governance-check.mjs",
  "business:evolution:check": "node scripts/business/evolution-sandbox-check.mjs",
  "business:security": "node scripts/business/security-controls.mjs"
}
```

Implement:

```
scripts/business/init-business.mjs
scripts/business/validate-business.mjs
scripts/business/eval-harness.mjs
scripts/business/governance-check.mjs
scripts/business/evolution-sandbox-check.mjs
scripts/business/security-controls.mjs
scripts/business/lib/schema-lite.mjs
scripts/business/lib/risk-tiers.mjs
scripts/business/lib/business-report.mjs
```

Use Node built-ins only.

9.1 `business:init`

Creates missing business directories and seed files without overwriting user content.

9.2 `business:validate`

Validates:

- business directory structure exists,
- schemas parse,
- examples conform to schemas,
- risk tiers are valid,

- workflow DNA has guardrails,
- high-risk workflows require human approval,
- irreversible actions require rollback plans,
- all rules/decisions/workflows include provenance fields where applicable.

9.3 `business:eval`

First version may be a dry-run evaluation harness.

It must:

- load evaluation cards,
- validate metric fields,
- report pass/fail/blocked,
- never promote a workflow automatically.

Support:

```
npm run business:eval -- --dry-run
```

9.4 `business:governance`

Checks for required governance artifacts:

- AI inventory,
- risk tiers,
- human approval policy,
- audit log folder,
- model card template,
- assurance case template,
- tool-permission register.

9.5 `business:evolution:check`

Ensures evolution artifacts do not mutate production directly.

It must fail if an approved/promoted production change is implied without:

- baseline comparison,
- regression test result,
- adversarial test result,
- compliance check,
- rollback plan,
- approval record where risk tier requires it.

9.6 `business:security`

Scans business artifacts for:

- accidental secrets,
- overly broad tool permissions,
- unsafe MCP/tool descriptions,
- missing human gates for sensitive actions,
- prompt-injection test coverage gaps.

This supplements, but does not replace, `npm run security`.

---

10. Trae IDE Outputs

Implement Trae-only workspace support.

The sync layer must generate or preview:

```
AGENTS.md
docs/agents.md
docs/trae.md
.trae/settings.json
.trae/rules/
.trae/agents/
.trae/commands/
```

Generated files must include:

```
<!-- AUTO-GENERATED by starter. Do not edit directly.
Source: rules/, skills/, hooks/, mcp-configs/
Run: npm run sync
-->
```

Create Trae-compatible rules for both the starter layer and the business layer.

Required starter rules from `starter.md`:

```
rules/00-constitution.md
rules/10-karpathy.md
rules/20-sdd.md
rules/30-security.md
rules/40-testing.md
rules/50-token-efficiency.md
rules/60-human-approval.md
```

Add business rules:

```
rules/70-business-operating-system.md
rules/80-process-intelligence.md
```

```
rules/90-governance-risk.md
rules/100-evolution-sandbox.md
rules/110-agentic-security.md
rules/120-evaluation-and-provenance.md
```

Create Trae agent definitions or generated agent docs for the roster in `structure.md`:

Control/meta agents:

- Business Orchestrator
- Evolution Manager
- Evaluation Harness
- Governance Officer
- Security Red-Team
- Memory Steward
- Tool Permission Broker
- Incident Commander

Learning agents:

- Expert Shadow
- Cognitive Task Analyst
- Process Miner
- Task Mining Agent
- Conformance Agent
- Bottleneck Analyzer
- Causal Improvement Agent
- Knowledge Distiller
- Knowledge Curator

Execution/domain agents may be added as examples:

- Quality Compliance Agent
- Execution Agent
- Finance Ops Agent
- Communications Agent

Each agent document should include:

- purpose,
- allowed actions,
- forbidden actions,
- risk-tier behavior,
- required evidence,
- memory read/write rules,
- human approval triggers,
- validation commands.

---

11. Skills and Commands

Create project skills under `skills/` with `SKILL.md` files where practical.

Required skill areas:

```
skills/planning/business-orchestration/SKILL.md
skills/implementation/workflow-dna/SKILL.md
skills/testing/evaluation-harness/SKILL.md
skills/review/governance-review/SKILL.md
skills/security/agentic-red-team/SKILL.md
skills/memory/memory-stewardship/SKILL.md
skills/lifecycle/evolution-sandbox/SKILL.md
skills/lifecycle/process-intelligence/SKILL.md
```

Create Trae command files or command docs under:

```
.trae/commands/
```

At minimum:

- bootstrap
- validate-business
- run-evals
- governance-check
- security-check
- sync-trae
- propose-evolution-variant
- review-source-audit

Commands must not execute downloaded third-party repo code.

---

12. Documentation Requirements

Create or update:

```
README.md
AGENTS.md
docs/installation.md
docs/usage.md
docs/agents.md
docs/trae.md
docs/architecture.md
docs/source-audit.md
docs/security.md
docs/sync.md
```

```
docs/troubleshooting.md
docs/changelog.md
docs/business-architecture.md
docs/process-intelligence.md
docs/knowledge-memory.md
docs/workflow-dna.md
docs/evolution-sandbox.md
docs/governance.md
docs/evaluation.md
```

The docs must explain:

1. How to run bootstrap.
2. How source download and audit work.
3. Why downloaded repos are untrusted.
4. How Trae sync works.
5. How the business operating-system structure maps to `structure.md`.
6. How event logs are captured.
7. How decision requirement cards are created.
8. How workflow DNA is represented.
9. How autonomy risk tiers work.
10. How human approvals are enforced.
11. How evolution proposals are sandboxed.
12. How evaluation cards and regression tests are used.
13. How security controls address LLM and agentic risks.
14. How to add a new workflow safely.
15. How to troubleshoot blocked downloads or validation failures.

---

13. Testing Requirements

Keep all tests using Node's built-in test runner.

Required starter tests from `starter.md`:

```
tests/manifest.test.mjs
tests/source-download.test.mjs
tests/source-audit.test.mjs
tests/sync.test.mjs
tests/adapters.test.mjs
```

Add business tests:

```
tests/business-structure.test.mjs
tests/business-schemas.test.mjs
tests/business-risk-tiers.test.mjs
tests/workflow-dna.test.mjs
```

```
tests/evolution-sandbox.test.mjs
tests/business-security.test.mjs
tests/business-governance.test.mjs
```

Tests must verify:

- `sources/manifest.json` parses.
- Every enabled source has required fields.
- No duplicate source IDs exist.
- Every source target starts with `external/sources/`.
- Disabled archived sources are not selected by default.
- `source-lock.json` shape is valid after real or mocked run.
- Generated files include the auto-generated header.
- `business/` structure exists.
- Business schemas parse.
- Example event logs validate.
- Example decision requirement cards validate.
- Example workflow DNA validates.
- High-risk workflow actions require human approval.
- Irreversible workflow actions require rollback.
- Evolution sandbox does not mutate production directly.
- Governance artifacts exist.
- Tool permissions are not overly broad by default.
- Security scanner detects obvious secret patterns in fixtures.

---

14. Bootstrap Flow

`npm run bootstrap` must run the starter bootstrap flow and then business validation.

For this repository's current state, the starter-only bootstrap phase has already completed successfully.

Do not rerun the starter bootstrap flow merely as part of continuing implementation work after starter completion.

Only rerun `npm run bootstrap` when:

- validating a changed starter layer,
- validating the integrated starter + business stack,
- or when the human explicitly asks for a full rerun.

Required order:

```
doctor
→ create required directories
→ validate sources/manifest.json
→ clone/update enabled GitHub sources
→ write sources/source-lock.json
→ generate docs/source-audit.md
```

```
→ run security smoke checks
→ run sync dry-run
→ run business initialization
→ run business validation
→ run business governance check
→ run business security check
→ run business evolution sandbox check
→ run tests
```

Implement this through `scripts/project-starter.mjs`.

If any required step fails, bootstrap must fail.

Optional source failures may be recorded and reported without failing unless strict mode is enabled.

---

15. Security and Safety Rules

Do not:

- execute downloaded repository code,
- run `npm install` inside downloaded repos,
- run remote install scripts,
- run `curl | bash`,
- install global packages,
- mutate global Trae configuration,
- copy third-party repo contents into active Trae config automatically,
- auto-enable MCP servers with credentials,
- store secrets in prompts, memory, or docs,
- overwrite user files without backup,
- delete files without explicit approval,
- promote evolution variants directly to production,
- log hidden chain-of-thought,
- auto-approve self-generated rules, skills, or workflow changes.

All third-party sources must remain:

```
external/sources/
```

and must be ignored by git.

Only curated, audited, attributed material may be copied into first-party directories, and only after human approval when high impact.

---

16. Acceptance Criteria

The implementation is accepted only when these commands pass or report exact blockers:

```
npm run doctor
npm run sources:download
npm run sources:audit
npm run security
npm run sync -- --dry-run
npm run business:init
npm run business:validate
npm run business:governance
npm run business:security
npm run business:evolution:check
npm run business:eval -- --dry-run
npm run test
```

After the business-layer implementation is ready, then run:

```
npm run bootstrap
```

If `starter.md` has already been executed successfully and no starter-layer files changed, do not treat another starter-only bootstrap rerun as mandatory interim work.

The following must exist after implementation:

```
package.json
starter.md
structure.md
sources/manifest.json
sources/docs-manifest.json
sources/source-lock.json
docs/source-audit.md
.trae/settings.json
.trae/rules/
.trae/agents/
.trae/commands/
business/
business/schemas/
business/examples/
business/governance/
business/security/
business/evolution/
```

Required downloaded source directories must exist under:

```
external/sources/
```

unless blocked by network/shell unavailability or a required repo failure, in which case the failure must be recorded in `sources/source-lock.json`, `docs/source-audit.md`, and `status.md`.

---

17. Status Updates

Update `status.md` at the end.

It must include:

- current phase,
- latest update,
- commands run,
- pass/fail result for each command,
- downloaded source count,
- failed source count,
- skipped source count,
- blockers,
- next steps.

If all checks pass, mark the project as bootstrap complete.

---

18. Final Report Format

When finished, report:

```
genetic-swarm-ops bootstrap complete.

Downloaded sources:
- <count> succeeded
- <count> failed
- <count> skipped

Generated:
- sources/source-lock.json
- docs/source-audit.md
- .trae/
- business/

Validation:
- doctor: pass/fail
- sources download: pass/fail
- source audit: pass/fail
- security: pass/fail
- sync dry-run: pass/fail
- business validation: pass/fail
- governance: pass/fail
- business security: pass/fail
- evolution sandbox check: pass/fail
- tests: pass/fail
```

```
Downloaded repositories are in external/sources/.
Generated Trae workspace files are under .trae/.
Business operating-system artifacts are under business/.
Downloaded repositories are not imported or executed until audited.
Evolution proposals cannot mutate production directly.
```

If incomplete, include exact blockers and the next command the human should run.

---

19. First Milestone Checklist

- [x] Read `starter.md` fully.
- [x] Read `structure.md` fully.
- [x] Create project structure.
- [x] Create `package.json`.
- [x] Create starter manifests.
- [x] Implement starter scripts.
- [x] Implement source downloader.
- [x] Implement source audit.
- [x] Implement Trae sync layer.
- [x] Create starter rules, skills, docs, tests.
- [x] Create `business/` structure.
- [x] Create business schemas.
- [x] Create business examples.
- [x] Create governance artifacts.
- [x] Create security artifacts.
- [x] Create evolution sandbox artifacts.
- [x] Implement business scripts.
- [x] Add business tests.
- [x] Run source downloads.
- [x] Generate source lock.
- [x] Generate source audit.
- [x] Run full validation.
- [x] Update `status.md`.
- [x] Report final result.